# MCP (Model Context Protocol): Revision Notes

## 1. What and why

- **MCP** = open standard that lets AI apps connect to external tools and data through one common interface.
- Analogy: **USB-C for AI**.
- **Problem before MCP:** every app needed custom code for every tool. M apps × N tools = **M × N integrations**.
- **After MCP:** each tool is wrapped once (server), each app is a client once. **M + N pieces**.
- Example: 3 apps + 4 tools → 12 integrations before, 7 pieces after. 10 apps + 20 tools → 200 vs 30.
- **Benefits:** build once and reuse everywhere, fix in one place, easy to swap model or app, ready-made server ecosystem, makes agents practical.

## 2. Architecture: 3 roles

| Role | Meaning | Example |
|---|---|---|
| **Host** | The AI app the user talks to | Claude Desktop, VS Code + Copilot, Cursor |
| **Client** | Lives inside the host, holds the connection to **one** server | Created automatically by the host |
| **Server** | Program exposing tools or data | GitHub server, DB server, your `server.py` |

- **One client per server.** Host with 3 servers = 3 clients.
- The **server has no AI**. It is a normal program exposing functions.
- The **model only decides** which tool to call. It **never runs code**. The client sends the call and the server executes it.
- The host controls which servers connect and what is allowed (approval prompts).

## 3. What a server can expose

| Type | Purpose | Controlled by | Analogy |
|---|---|---|---|
| **Tool** | Function the AI can **run** (action) | **Model** | Waiter cooks something |
| **Resource** | **Read-only** data, identified by a URI | **App** | The menu card |
| **Prompt** | Reusable prompt template | **User** | Combo meal |

- Examples: tool `create_ticket`; resource `file:///logs/app.log`; prompt `review_code`.
- Most servers are mostly **tools**.
- The model picks a tool from its **name** + **docstring/description** + **input schema**. Docstrings and type hints are what the model sees.

## 4. The protocol: JSON-RPC 2.0

### 4. The protocol: JSON-RPC 2.0

- **RPC** = Remote Procedure Call (call a function that lives elsewhere). **JSON-RPC** = rules for doing that with JSON messages.
- Request: `method`, `params`, `id`. Response: `result` (or `error`) with the **same id**.
- **Notification** = message with no reply (no id).

| Method | Purpose |
|---|---|
| `initialize` | Handshake |
| `tools/list`, `tools/call` | Discover and run tools |
| `resources/list`, `resources/read` | Discover and read resources |
| `prompts/list`, `prompts/get` | Discover and fetch prompts |
| `notifications/tools/list_changed` | Server says tool list changed |

### 5. Transports (how messages travel)

| | stdio | Streamable HTTP |
|---|---|---|
| How | Client launches server as child process, talks via stdin/stdout | Server runs at a URL (e.g. `/mcp`), client sends HTTP requests |
| Server started by | The client | You (already running) |
| Use for | Local, on your machine | Remote or shared servers |
| Auth | Not needed (local); secrets via env vars | **Required** (OAuth 2.1 in the spec) |

- Same JSON-RPC messages in both. Only the transport changes.
- **stdio rule:** never `print()` to stdout (it corrupts the messages). Log to **stderr**.
- An HTTP MCP server is not a website. Opening it in a browser gives "Not Found" or an error, and that is normal.

### 6. End-to-end flow

1. **Startup:** host launches the server (stdio: `python server.py`).
2. **Handshake:** client sends `initialize` (protocol version, client info, capabilities). Server replies with its version, info, and capabilities. Client sends `notifications/initialized`.
3. **Discovery:** client calls `tools/list` (also resources and prompts). **The client learns the tools here, before the user asks anything.**
4. **User asks a question.**
5. **Host → model:** question + tool list.
6. **Model decides** and returns a tool call (name + arguments).
7. **Approval:** host usually asks the user to Allow.
8. **Execution:** client sends `tools/call`, server runs the function and returns the result.
9. **Host → model:** result goes back. Steps 6 to 9 repeat if more tools are needed.

10. **Model writes the final answer.**

11. **Shutdown:** connection closes, server process exits.

One-liner: `launch → initialize → list tools → user asks → model picks tool → tools/call → result → model answers`

### 5b. Building a server (Python SDK, key points)

- Install: `pip install "mcp[cli]"`.
- `FastMCP("name")` creates the server (high-level, handles JSON-RPC and handshake).
- Decorators: `@mcp.tool()`, `@mcp.resource("uri")`, `@mcp.prompt()`.
- `mcp.run(transport="stdio")` or `"streamable-http"`.
- **Version gotcha:** mcp **2.x renamed** `FastMCP` to `MCPServer` (`from mcp.server.mcpserver import MCPServer`). Pin `mcp<2` to keep the 1.x API.

### 7. Testing and clients

- **MCP Inspector:** test client with a web UI. Run `npx @modelcontextprotocol/inspector python server.py`.
  - Tabs: Tools, Resources, Prompts match the 3 capabilities.
  - **Protocol/Messages panel** = raw JSON-RPC. **Console** = server logs and errors.
  - A local proxy launches the server over stdio, and the browser talks to the proxy. The URL token stops other pages from controlling it.
- `mcp dev` needs `uv`. Without it, use the Inspector command above.
- **Own client (Python SDK):** `stdio_client` (launch), `ClientSession` (JSON-RPC), then `initialize()`, `list_tools()`, `call_tool(name, args)`, `read_resource(uri)`.
- Real hosts: Claude Desktop (config `claude_desktop_config.json`), VS Code (`.vscode/mcp.json`). Use **full paths** and the venv's Python.
- Quick proof a real tool ran: use a tool returning a value the model can't guess.

### 8. Security (high-value interview topic)

| Risk | Meaning | Defense |
|---|---|---|
| **Tool poisoning** | Malicious instructions hidden in tool descriptions | Only use trusted servers |
| **Prompt injection** | Tool output (web page, email) carries instructions | Treat returned data as untrusted |
| **Over-powered tools** | e.g. `run_sql` with admin rights | **Least privilege**: read-only users, specific tools (`get_order(id)`) |
| **No human check** | Destructive actions run silently | Keep **approval** for delete, send, pay |
| **Leaked secrets** | Keys hardcoded in code | Env variables, never in Git |
| **Bad inputs** | Model can pass wrong or tricked | Validate inputs, use parameterized SQL |

| | | |
|---|---|---|
| | arguments | |
| **Open HTTP server** | Anyone can run your tools | Authentication (OAuth) |

## 9. Good server design

- Small, focused tools (not one `do_everything`).
- Clear docstrings (the description decides whether the model picks the tool).
- Short, useful output. Don't dump thousands of rows.
- Clear error messages (raise exceptions) so the model can recover.
- Don't expose dozens of tools. Group by purpose, one server per area.

## 10. Likely interview questions

**What is MCP and why was it created?** Open standard connecting AI apps to tools and data. It replaces M × N custom integrations with M + N standard pieces.

**Explain host, client, and server.** Host = AI app. Client = connector inside the host, one per server. Server = exposes tools, resources, prompts.

**Tool vs resource vs prompt?** Tool = action, model-controlled. Resource = read-only data, app-controlled. Prompt = template, user-controlled.

**How does the client know what tools exist?** `tools/list` right after the handshake. The list (names, descriptions, schemas) is passed to the model with the user's question.

**Does the model execute the tool?** No. The model only chooses the tool and arguments. The client sends `tools/call` and the server runs it.

**Why JSON-RPC?** It gives rules on top of JSON: method, params, id, result or error, and notifications.

**stdio vs HTTP?** stdio = local child process, no auth. HTTP = remote and shared, needs auth. Same messages.

**How is MCP different from a normal REST API?** REST is per-service with its own format. MCP is one standard with built-in **discovery** (`tools/list`) so an AI can find and use tools without custom code. It is not a replacement for REST. Many servers wrap REST APIs.

**What are the security risks?** Tool poisoning, prompt injection, over-privileged tools, leaked secrets, unauthenticated servers. Use least privilege, approval, validation, and trusted servers only.

**Why must a stdio server not use `print()`?** Stdout carries the JSON-RPC messages, so stray output corrupts the protocol. Log to stderr.

**How would you design a good MCP tool?** Single purpose, clear docstring, typed arguments, short output, helpful errors, least privilege.

## 11. Quick facts

- Handshake only declares **capabilities**. Actual tool lists come from `tools/list`.
- `id` matches a response to its request.
- Model can answer simple questions without tools. To force a tool, name it or use a value only the tool knows.
- Task tracker project: tools (`add_task`, `list_tasks`, `complete_task`) + resource (`tasks://summary`) +

prompt ( `plan_my_day` ) + SQLite for persistence.